ARQUITETURA DE SOFTWARE

CAPÍTULO 1 - QUAIS SÃO OS FUNDAMENTOS E CONCEITOS BÁSICOS DA ARQUITETURA DE *SOFTWARE* ?

Fernando Skackauskas Dias

INICIAR

Introdução

A complexidade dos sistemas de *software* tem aumentado consideravelmente nas últimas décadas devido à inclusão de novas interfaces, integração de várias mídias e novas tecnologias de armazenamento e distribuição de dados. Nesse sentido, os engenheiros de *software* têm utilizado novas abordagens, a fim de desenvolver sistemas com alto desempenho. Portanto, desenvolver *softwares* com qualidade é uma questão que tem merecido a devida atenção dos cientistas da computação.

Assim, podemos nos questionar: qual o papel da arquitetura de *software*? Quais são os modelos de arquitetura? Como a implantação de um modelo de arquitetura de *software* pode melhorar a qualidade dos sistemas de informação?

Antes de responder a essas questões, é importante relembrarmos que a arquitetura de *software* é considerada a área de conhecimento da Ciência da Computação na qual são aplicadas práticas da engenharia de *software* e da gerência de projetos, procurando obter melhor organização dos sistemas e com alta produtividade. São nas bases dessas disciplinas é que encontramos os modelos para especificar, projetar, implantar e manter sistemas com qualidade.

Iniciaremos esta disciplina analisando os princípios de arquitetura de *software* com a demonstração das fases de desenvolvimento de um projeto arquitetural e a descrição dos elementos pertinentes à arquitetura e os padrões arquiteturais básicos. A seguir, explicaremos os tipos de modelagem e a utilização dos diferentes estilos, padrões e gêneros na especificação da arquitetura de *software*. Por fim, analisaremos os impactos das decisões do tipo de arquitetura no desempenho de um sistema.

Desejamos um excelente estudo.

1.1 Introdução à arquitetura de *software*

Você sabe qual o propósito da arquitetura de *software*? Segundo Galloti (2016, p. 3), é o de "criar soluções [...] cada vez mais simples, rápidas e eficientes". Além do exposto pelo autor, devemos ir além, ou seja, não somente resolver problemas, mas também **prevenir** a ocorrência deles e **aperfeiçoar** as soluções já existentes.

A história recente da Ciência da Computação mostra a evolução das técnicas e dos métodos de desenvolvimento de sistemas, passando pela programação estruturada, orientada a evento até a orientação a objeto, assim como a evolução da lógica dos algoritmos e da estrutura de dados.

Diante da complexidade dos sistemas atuais, é inevitável que tenhamos estilos de arquitetura acompanhando tal evolução. É necessário ter um conhecimento bastante sólido dos fundamentos e conceitos básicos da arquitetura de *software* para compreender a evolução dos sistemas de informação e a estruturação lógica e abstrata deles, além de desenvolvê-los com confiabilidade e alta qualidade.

1.1.1 Conceitos introdutórios de arquitetura de software

Qual o sentido fundamental de arquitetura de *software*? Para Galloti (2016, p. 10), ela "[...] explica a forma como [ele] se organiza e funciona, além de seu modo de implementação". Portanto, agrega os componentes denominados elementos arquiteturais (dados, processamentos e conexão), que se organizam de maneira lógica para atender aos requisitos funcionais e não funcionais. Para tal, a arquitetura de *software* deve ser clara, a fim de atingir o máximo da simplicidade, tendo o *software* as seguintes características:

- **flexível**: deve permitir as mudanças que se façam necessárias em decorrência das alterações de requisitos;
- **extensível**: deve ter a capacidade de incorporar novos elementos, permitindo que a estrutura do sistema como um todo se estenda;
- **portabilidade**: permite que ele seja executado em mais de uma plataforma;
- **reutilizável**: é possível adaptar componentes de um *software* para outro.

O **projeto de arquitetura** é o primeiro passo no processo de implantação de um modelo de arquitetura de *software*: ele a conecta com a **engenharia de requisitos** – esta última é o processo que elabora os documentos dos requisitos funcionais e não funcionais durante todo o ciclo de vida do sistema. É nesta parte do processo que são identificados os componentes estruturais e os seus relacionamentos.

A arquitetura de *software* se altera na mesma medida em que ele evolui e se ajusta às novas demandas e evoluções tecnológicas. Portanto, ela é parte integrante da **engenharia de *software***, que é uma abordagem sistemática e formal de desenvolvimento dos sistemas de informação.

Essas características fazem parte dos **quesitos de qualidade**, ou seja, é o conjunto de atributos que se espera de um *software* de alto desempenho. Podemos detalhá-las no quadro a seguir, segundo Sommerville (2011, p. 5).

Características do produto	Descrição
Manutenibilidade	O *software* deve ser escrito de forma que possa evoluir para atender às necessidades dos clientes. Esse é um atributo crítico, porque a mudança de *software* é um requisito inevitável de um ambiente de negócio em mudança.
Confiança e proteção	A confiança do *software* inclui uma série de características como confiabilidade, proteção e segurança. Um *software* confiável não deve causar prejuízos físicos ou econômicos no caso de falha de sistema. Usuários maliciosos não devem ser capazes de acessar ou prejudicar o sistema.
Eficiência	O *software* não deve desperdiçar os recursos do sistema, como memória e ciclos do processador. Portanto, eficiência inclui capacidade de resposta, tempo de processamento, uso de memória etc.
Aceitabilidade	O *software* deve ser aceitável para o tipo de usuário para o qual foi projetado. Isso significa que deve ser compreensível, usável e compatível com outros sistemas usados por ele.

Quadro 1 - Atributos essenciais de qualidade de um software, sendo parte integrante da elaboração da arquitetura de um sistema de informação. Fonte: SOMMERVILLE, 2011, p. 5.

#PraCegoVer: Contém uma tabela com os atributos essenciais de qualidade de um software, sendo parte integrante da elaboração da arquitetura de um sistema de informação. A tabela é composta de duas colunas denominadas, Características do produto e Descrição. A coluna Características do produto contém as linhas, sendo elas: Manutenibilidade, Confiança e proteção, Eficiência e, Aceitabilidade. A descrição de Manutenibilidade é: O software deve ser escrito de forma que possa evoluir para atender às necessidades dos clientes. Esse é um atributo crítico, porque a mudança de software é um requisito inevitável de um ambiente de negócio em mudança. A descrição de Confiança e proteção é: A confiança do software inclui uma série de características como confiabilidade, proteção e segurança. Um software confiável não deve causar prejuízos físicos ou econômicos no caso de falha de sistema. Usuários maliciosos não devem ser capazes de acessar ou prejudicar o sistema. A descrição de Eficiência é: O software não deve desperdiçar os recursos do sistema, como memória e ciclos do processador. Portanto, eficiência inclui capacidade de resposta, tempo de processamento, uso de memória, entre outros. A descrição de Aceitabilidade é: O software deve ser aceitável para o tipo de usuário

para qual foi projetado. Isso significa que deve ser compreensível, usável e compatível com outros sistemas usados por ele.

Estas características terão maior ou menor relevância dependendo do propósito do *software* e do contexto no qual ele está inserido. Por exemplo, *softwares* de transações informacionais básicas (como, por exemplo, um sistema de cadastro de clientes de uma loja) terão atributos de qualidade diferentes de sistemas mais complexos como, por exemplo, um *software* de controle de uma fábrica. No primeiro caso, o foco do sistema estará mais atrelado à interface do cliente, e fatores como usabilidade e acessibilidade serão mais críticos. Por outro lado, nos sistemas de controle industrial, por exemplo, os atributos de confiança e proteção serão mais críticos devido à natureza e finalidade deste sistema.

Nesse sentido, existem abordagens para que os *softwares* sejam tipificados conforme sua **aplicabilidade** e **natureza**, sendo fundamentais ao construirmos o projeto de arquitetura de um sistema, pois estes fatores impactam quando modelamos o desenho dos componentes e suas relações com o fluxo de dados nos quais haverá alguma interação.

Como explicamos, as abordagens de desenvolvimento variam conforme o objetivo do sistema e o ambiente no qual ele é desenvolvido. O fator mais importante para determinar quais são as técnicas e os métodos de engenharia de *software* é o **tipo de aplicação** que, segundo Sommerville (2011, p. 7, grifos nossos), podem ser identificados como:

- **Aplicações *stand-alone***. Essas são as aplicações executadas em um computador local, como um PC. Elas contêm toda a funcionalidade necessária e não precisam estar conectadas a uma rede.
- **Aplicações interativas baseadas em transações**. São aplicações que executam em um computador remoto, acessadas pelos usuários a partir de seus computadores ou terminais. Certamente, aqui são incluídas aplicações *Web* como aplicações de comércio.
- **Sistemas de controle embutidos**. São sistemas de controle que controlam e gerenciam dispositivos de *hardware*. Exemplos de sistemas embutidos é o *software* em telefone celular ou que controlam antitravamento de freios em um carro.
- **Sistemas de processamento de lotes**. São sistemas corporativos projetados para processar dados em grandes lotes. Exemplos de sistemas de lotes incluem sistemas periódicos de cobrança, como sistemas de cobrança telefônica, e sistemas de pagamentos de salário.

- **Sistemas de entretenimento**. São sistemas cuja utilização principal é pessoal e cujo objetivo é entreter o usuário.
- **Sistemas para modelagem e simulação**. São sistemas que incluem vários objetos separados que interagem entre si, desenvolvidos por cientistas e engenheiros para modelar processos ou situações físicas.
- **Sistemas de coleta de dados**. São sistemas que coletam dados de seu ambiente com um conjunto de sensores e enviam esses dados para outros sistemas para processamento.
- **Sistemas de sistemas**. São sistemas compostos de uma série de outros sistemas de *software*. Alguns deles podem ser produtos genéricos de *software*, como um programa de planilha eletrônica.

Tendo como base as tipificações definidas anteriormente, é necessário definir como são classificados os **modelos** de processo de desenvolvimento de *software*, isto é, a representação, de forma simplificada, de um determinado processo específico. Segundo Sommerville (2011, p. 20, grifos nossos), os modelos são englobados em três tipos:

- **em cascata**. Este modelo determina as atividades principais do processo de especificação, desenvolvimento, validação e evolução, sendo que cada uma das partes do modelo são consideradas fases distintas.
- **desenvolvimento incremental**. Este modelo integra as atividades de especificação, desenvolvimento e validação, porém é desenvolvido como sendo uma série de versões - ou incrementos - sendo que cada versão agrega novas funcionalidades à versão anterior.
- **engenharia de *software* orientada a reuso**. Esta abordagem considera um número considerável de componentes reusáveis.

A partir das tipificações de sistemas e dos modelos de processo de desenvolvimento, são definidos os **níveis** e aquilo que é considerado a **decomposição** da arquitetura de *software* – esta, segundo Sommerville (2011, p. 104), é necessária para organizar e estruturar a especificação da arquitetura do sistema, sendo útil no momento em que se realiza o levantamento dos requisitos junto aos envolvidos no desenvolvimento do sistema. Podemos projetar a arquitetura de *software* em dois níveis de abstração, segundo Sommerville (2011, p. 104, grifos nossos):

- A arquitetura em **pequena escala** está preocupada com a arquitetura de programas individuais. Nesse nível, estamos preocupados com a maneira como um programa individual é decomposto em componentes.

- A arquitetura em **grande escala** preocupa-se com a arquitetura de sistemas corporativos complexos que incluem outros sistemas, programas e componentes de programas. Esses sistemas empresariais estão distribuídos por diversos computadores, que podem pertencer e ser geridos por diferentes empresas

Desenhar a arquitetura de *software* é fundamental, portanto, para o **desenvolvimento** de um sistema, pois integra o desempenho, a robustez e a capacidade do sistema de sua manutenibilidade.

VOCÊ QUER LER?

Existem arquiteturas de *softwares* com diversas abordagens, inclusive as que são orientadas para a avaliação do processo de ensino-aprendizagem de forma contínua. Esta, inclusive, é bastante criativa e interessante, e foi desenvolvida em uma pesquisa feita pelos autores Cunha; Filgueira; Viana (2017). Leia mais em: <https://www2.ifrn.edu.br/ojs/index.php/HOLOS/article/view/5335/pdf (https://www2.ifrn.edu.br/ojs/index.php/HOLOS/article/view/5335/pdf)>.

As arquiteturas de *software* geralmente são modeladas usando **diagramas de blocos** simples, nos quais cada caixa representa um componente. Quando são representadas caixas dentro de caixas significa que o componente foi decomposto em subcomponentes. Por outro lado, as setas indicam que os dados são passados de um componente ao outro. Um exemplo de representação de uma arquitetura de *software* utilizando diagrama de blocos é representado na figura a seguir, que elabora um controle robotizado de empacotamento.

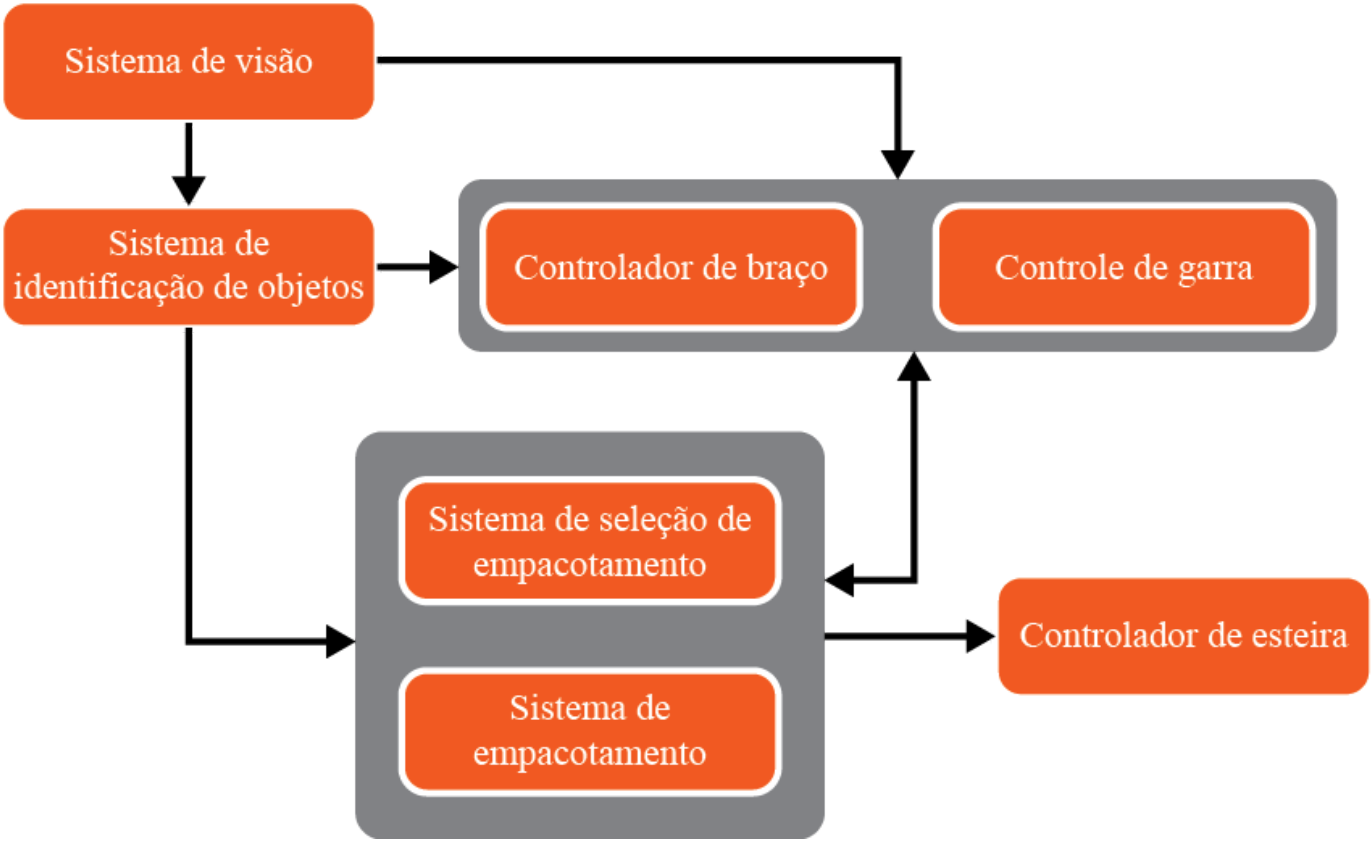


Figura 1 - Arquitetura de um sistema de controle robotizado de empacotamento, indicando os componentes, os subcomponentes e a transação de dados. Fonte: Elaborada pelo autor, adaptado de SOMMERVILLE, 2011.

#PraCegoVer: Ilustra a arquitetura de um sistema de controle robotizado de empacotamento, indicando os componentes, os subcomponentes e a transação de dados. A ilustração contém o componente Sistema de Visão que aponta para o componente Sistema de identificação de objetos e para os subcomponentes Controlador de Braço e Controle de Garra. Estes subcomponentes também são apontados pelo componente Sistemas de Identificação de Objetos, que por sua vez, também aponta para os subcomponentes Sistema de Seleção de Empacotamento e, Sistema de Empacotamento. Estes subcomponentes apontam para o componente Controlador de Esteira.

Os diagramas de blocos representam a arquitetura de um *software* e apresentam uma visão de alto nível da estrutura do sistema, sendo que os usuários de diferentes áreas que estão envolvidos no processo de desenvolvimento podem compreender a organização dos componentes, suas relações e a transação de dados.

1.1.2 Descrições de arquitetura

Quais são as visões e descrições que são necessárias ao se projetar uma arquitetura de sistema? Quais notações devem ser usadas com o objetivo de se descrever os modelos de arquitetura? É necessário considerar que é impossível representar todas as informações relevantes da arquitetura de *software* em um único modelo, porque cada modelo representa uma perspectiva do sistema. Portanto, são necessários vários deles para conceber a arquitetura na sua totalidade. Os autores da área de arquitetura e engenharia de *software* propõem que devemos ter **quatro visões** fundamentais, as quais, segundo Sommerville (2011, p. 107, grifos nossos), são:

- **lógica**, que mostra as abstrações fundamentais do sistema como objetos ou classes de objetos. Nessa visão, deveria ser possível relacionar os requisitos de sistema com as entidades.
- **de processo**, que mostra como, no tempo de execução, o sistema é composto de processos interativos. Essa visão é útil para fazer julgamentos sobre as características não funcionais do sistema, como desempenho e disponibilidade.
- **de desenvolvimento**, que mostra como o *software* é decomposto para o desenvolvimento, ou seja, apresenta a distribuição do *software* em componentes que são implementados por um único desenvolvedor ou por uma equipe de desenvolvimento. Essa visão é útil para gerentes de *software* e programadores.
- **física**, que mostra o *hardware* do sistema e como os componentes de *software* são distribuídos entre os processadores. Essa visão é útil para os engenheiros de sistemas que estão planejando uma implantação do sistema.

Hofmeister; Nord; Soni (2000) argumentam que é necessário, também, incluir uma **visão conceitual**, pois ela pode servir como uma decomposição dos requisitos de um nível mais alto para um mais detalhado. Essa ferramenta é fundamental para a administração da complexidade do sistema, porque pode esconder detalhes, permitindo que os desenvolvedores se concentrem no nível de abstrações do sistema.

VOCÊ SABIA?

A internet das coisas é uma revolução tecnológica recente que tem como objetivo conectar os itens – ou seja, as coisas – utilizados no dia a dia com a internet. Cagnin (2015) desenvolveu sua dissertação propondo uma arquitetura de *software* denominada de "Uma arquitetura multiagente para o gerenciamento de dispositivos em ambientes da internet das coisas". Veja mais em: <http://www.athena.biblioteca.unesp.br/exlibris/bd/cathedra/02-08-2017/000868971.pdf (http://www.athena.biblioteca.unesp.br/exlibris/bd/cathedra/02-08-2017/000868971.pdf)>.

Uma das possíveis visões que podem servir de referência para a construção da arquitetura de um sistema é a *Unified Modeling Language* (**UML**), uma linguagem de modelagem que serve para definir artefatos que auxiliam na tarefa de desenhar e documentar os sistemas, sendo composta por **diversos diagramas** que compõem a estrutura do projeto de arquitetura do sistema. Segundo Galloti (2016, p. 81, grifos nossos), os diagramas UML são de:

- **Caso de Uso**. Este diagrama define o grupo de comportamentos no alto nível do sistema e como deve ser executado para um determinado ator.
- **Classes**. Este diagrama representa a coleção de classes e define seus inter-relacionamentos.
- **Objetos**. Este diagrama visa mostrar em forma de um retrato os objetos do *software* e seus inter-relacionamentos.
- **Colaboração**. Este diagrama figura uma coleção de objetos que trabalham conjuntamente para atender o comportamento do sistema.
- **Sequência**. Representa uma perspectiva que é orientada por tempo, da colaboração existente entre os objetos.
- **Atividades**. Este diagrama mostra o fluxo das tarefas que são executadas pelo sistema ou por um determinado ator.
- **Estados**. Este diagrama mostra um conjunto de estados de um objeto pode ter e os 'gatilhos' que fazem a transição do objeto de um determinado estado para outro.
- **Componentes**. Representa uma coleção de componentes pertinentes ao sistema e seus inter-relacionamentos.
- **Depuração**. Este diagrama mostra um conjunto de componentes e como eles são distribuídos nos nós de *hardware*.
- **Pacotes**. Mostra uma coleção de outros possíveis elementos de modelagem ou de diagramas.

Alguns autores têm opiniões divergentes pelo uso da UML para a construção da arquitetura de *software*, pois às vezes ela é usada de forma inadequada para demonstrar o comportamento dos componentes do sistema e suas relações. Sua utilização – ou não – dependerá do nível de experiência do desenvolvedor e da complexidade do sistema que se está implementando. Por outro lado, vários pesquisadores indicam o uso de linguagens de descrição de arquitetura

(*Architectural Description Languages*, as **ADLs**) para apresentá-la. Os elementos fundamentais das ADLs são os **componentes** e **conectores**, que agregam regras e diretrizes para a arquitetura do sistema.

1.2 Gêneros e estilos de arquitetura

Podemos conceituar o projeto de arquitetura, segundo Sommerville (2011, p. 105), como "[...] um processo criativo no qual você projeta uma organização de sistema para satisfazer aos requisitos funcionais e não funcionais de um sistema". Baseando-nos na afirmativa do autor, podemos perguntar: existe uma arquitetura genérica que pode atuar como um modelo para o sistema que está sendo projetado? Quais gêneros ou estilos de arquitetura podem ser usados?

O que temos é que, durante o **processo de projeto** de arquitetura, os desenvolvedores precisam tomar várias decisões estruturais que podem afetar de forma significativa o sistema e todo o seu processo de desenvolvimento. Segundo Sommerville (2011, p. 106), "um padrão de arquitetura é uma descrição abstrata de boas práticas vivenciadas e testadas em diferentes ambientes e sistemas", logo ele deve descrever a organização de um sistema que foi bem-sucedida e incluir informações de quando o uso desse padrão é adequado, incluindo os pontos fortes e fracos. A seguir, descreveremos os padrões existentes na arquitetura de *software* com exemplos e situações em que são utilizados, bem como suas vantagens e desvantagens.

A arquitetura de um *software* pode ser baseada em um determinado padrão ou estilo. Um **padrão de arquitetura** significa como o *software* é organizado: por exemplo, existe o padrão de organização cliente-servidor e um padrão de arquitetura em camadas. Esses padrões mostram o objetivo de uma arquitetura que foi utilizada em sistemas de *software* diferenciados. Nesse sentido, o desenvolvedor, ao tomar decisões sobre qual a arquitetura de um sistema, deve conhecer os padrões mais comuns e saber onde eles podem ser usados e quais são seus prós e contras.

É necessário que o desenvolvedor saiba escolher a estrutura que mais se adequa às necessidades do sistema a ser desenvolvido, como a arquitetura cliente-servidor ou a em camadas, que seja capaz de permitir o atingimento dos **requisitos do sistema**. Para que se decomponham em unidades estruturais do sistema, o desenvolvedor

deverá decidir sobre qual a estratégia utilizada para a decomposição de componentes em seus subcomponentes. As abordagens que podem ser usadas devem permitir a implantação de tipos distintos de arquitetura.

Por fim, ao chegar ao processo de modelagem de controle, é necessário tomar decisões sobre como a **execução dos componentes** do sistema é controlada. Ou seja, se desenvolve um modelo de visão mais alta dos relacionamentos de controle entre as diversas unidades do sistema. Segundo Sommerville (2011, p. 106, grifos nossos), devem ser analisados os seguintes requisitos para que se faça a escolha do **estilo de arquitetura**:

- **Desempenho**. Se o desempenho for um requisito crítico, a arquitetura deve ser projetada para localizar as operações críticas dentro de um pequeno número de componentes.
- **Proteção**. Se a proteção for um requisito crítico, deve ser usada uma estrutura em camadas para a arquitetura, com os ativos mais críticos protegidos nas camadas mais internas.
- **Segurança**. Se a segurança for um requisito crítico, a arquitetura deve ser concebida de modo que as operações relacionadas com a segurança estejam localizadas em um único componente.
- **Disponibilidade**. Se a disponibilidade for um requisito crítico, a arquitetura deve ser projetada para incluir componentes redundantes.
- **Manutenção**. Se a manutenção for um requisito crítico, a arquitetura do sistema deve ser projetada a partir de componentes autocontidos de baixa granularidade que podem ser rapidamente alterados.

Para Pressman (2016), apesar de os princípios do projeto de arquitetura de *software* se aplicarem a todos os tipos de sistemas, o gênero ditará a abordagem específica para a estrutura que deve ser construída, isto é, uma categoria específica no domínio de *software* de forma geral, sendo que em cada categoria podem ser enquadradas subcategorias. Segundo Pressman (2016, p. 233, grifos nossos), podemos ter os seguintes **gêneros de arquitetura**:

- **Inteligência artificial** – sistemas que simulam a cognição.
- **Comercial e sem fins lucrativos** – sistemas para operação de empreendimento.
- **Comunicações** – sistema para infraestrutura e gerenciamento de dados.
- **Autoria de conteúdo** – sistemas para controlar artefatos.
- **Dispositivos** – sistemas que interagem com o mundo físico.
- **Esportes e entretenimento** – sistemas que gerenciam eventos públicos.
- **Financeiros** – sistemas para gerenciar movimentações financeiras.

- **Jogos** – sistemas que geram experiência de entretenimento.
- **Governo** – sistemas que dão apoio às entidades governamentais.
- **Industriais** – sistemas que controlam processos físicos.
- **Legais** – sistemas que dão apoio jurídico.
- **Médicos** – sistemas de diagnóstico.
- **Militar** – sistemas de controle de inteligência militar.
- **Sistemas operacionais** – sistemas que se situam acima do *hardware*.
- **Plataformas** – sistemas que posicionam acima dos sistemas operacionais.
- **Científicos** – sistemas utilizados para pesquisa.
- **Ferramentas** – sistemas utilizados para desenvolver outros sistemas.
- **Transportes** – sistemas que controlam veículos.
- **Serviços públicos** – sistemas para oferecer serviço à sociedade.

Por outro lado, a questão de **estilos** na arquitetura de *software* descreve uma categoria de sistema: pode englobar um conjunto de componentes que realiza uma função que é exigida, ou diversos conectores e várias restrições que definem como os componentes interagirão. Seu objetivo é estabelecer uma estrutura única para todos os componentes do sistema e, de acordo com Pressman (2016), eles podem ser classificados como:

- **arquitetura centralizada em dados**: um repositório de dados reside no centro desta arquitetura e é, em geral, acessado por outros componentes. Observe-a na figura a seguir.

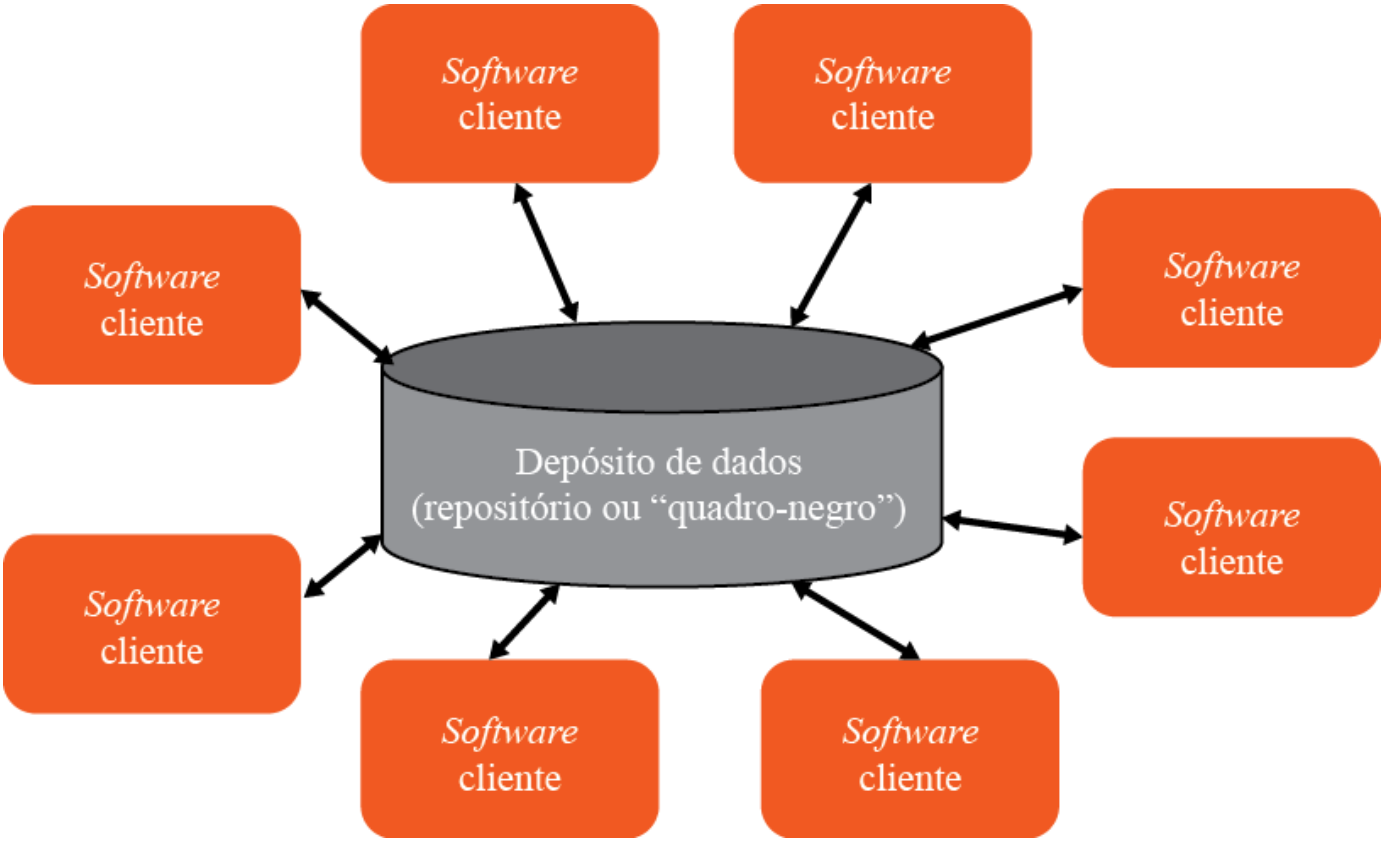


Figura 2 - Arquitetura centralizada em dados demonstrando o depósito de dados e os softwares associados.

Fonte: Elaborada pelo autor, baseado em PRESSMAN, 2016.

#PraCegoVer: Ilustra a arquitetura centralizada em dados demonstrando o depósito de dados e os softwares associados. Ao centro existe um cilindro, representando o depósito de dados e ao redor, vários retângulos, representando os softwares que consomem e abastecem o depósito de dados.

- **arquitetura de fluxo de dados**: dados de entrada devem ser transformados por meio de uma série de componentes. Perceba-a na figura a seguir.

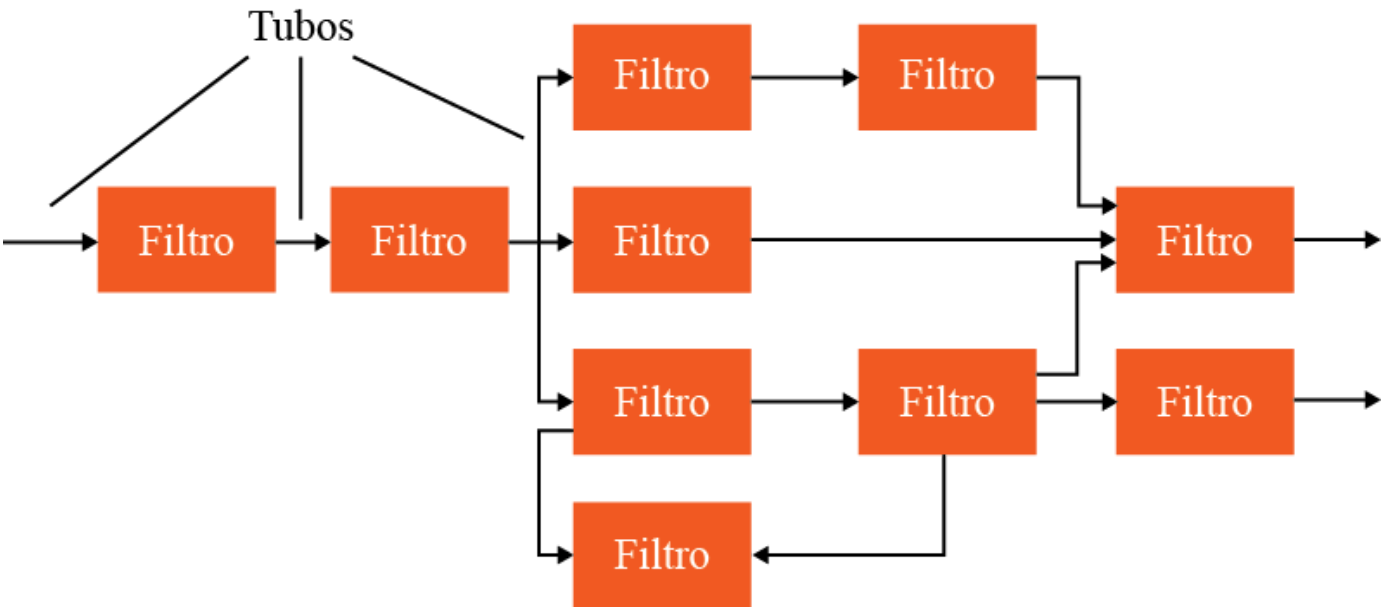


Figura 3 - Arquitetura de fluxo de dados demonstrando os filtros de dados e os tubos associados. Fonte: Elaborada pelo autor, baseado em PRESSMAN, 2016.

#PraCegoVer: Contém a ilustração da Arquitetura de fluxo de dados demonstrando os filtros de dados e os tubos associados. A imagem é composta por vários retângulos, contendo em seu interior a palavra Filtro, onde vários filtros apontam para vários filtros. As ligações entre estes filtros, são chamadas de tubos.

- **arquitetura de chamadas e retornos**: permite obter dados em programa principal e subprograma. Atente-a na figura a seguir.

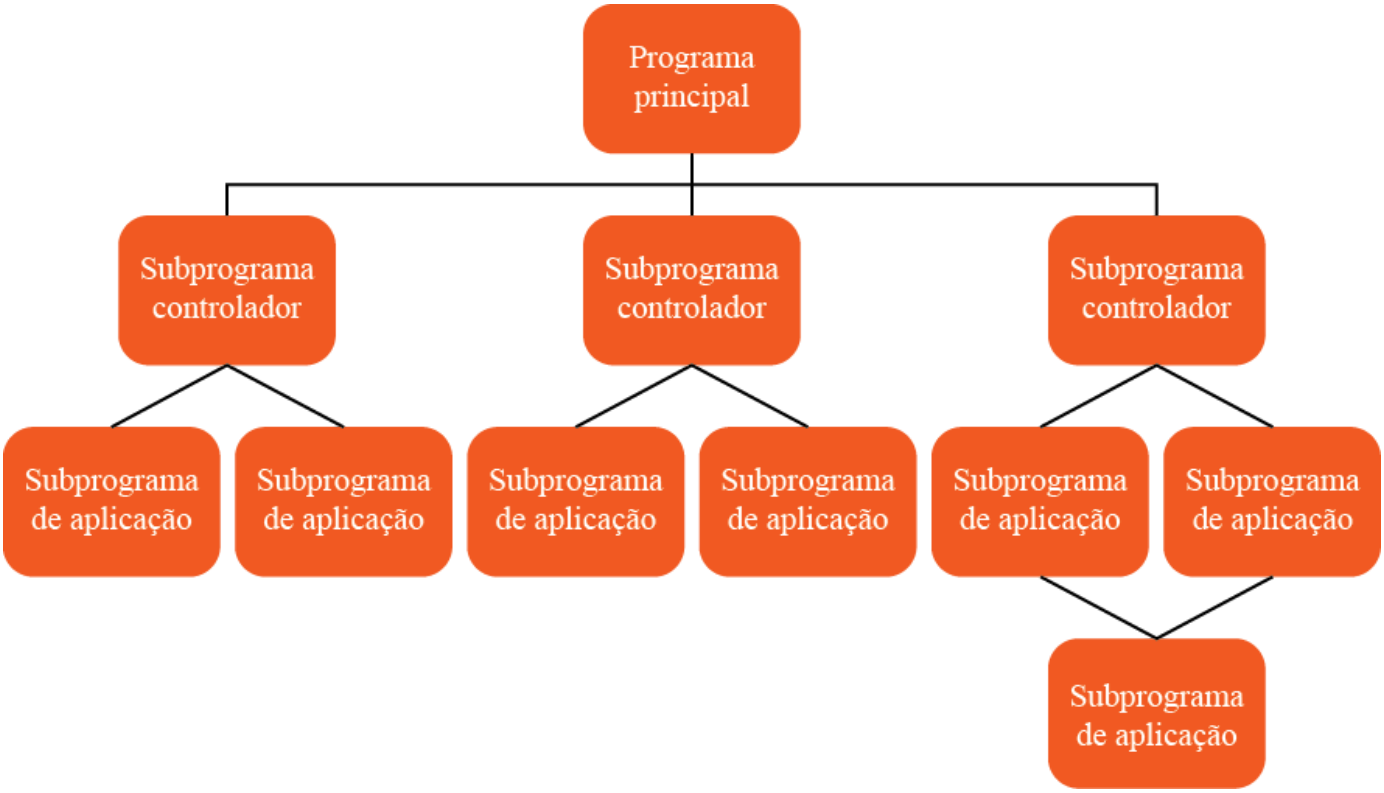


Figura 4 - Arquitetura de chamadas e retornos demonstrando o programa principal e subprogramas controladores e de aplicação. Fonte: Elaborada pelo autor, baseado em PRESSMAN, 2016.

#PraCegoVer: Ilustra a arquitetura de chamadas e retornos, demonstrando o programa principal e subprogramas controladores e de aplicação. A imagem é composta por vários retângulos. Inicia com um retângulo contendo a inscrição Programa Principal. Em um nível inferior, há três retângulos contendo a inscrição Subprograma de Aplicação, que estão ligados ao Programa Principal e também, a outros subprogramas.

- **arquitetura orientada a objetos**: caracterizada pelos componentes terem os dados e as operações encapsulados.
- **arquitetura em camadas**: sua estrutura básica é definida em camadas distintas, que interagem em operações de forma progressiva. Observe-a na figura a seguir.

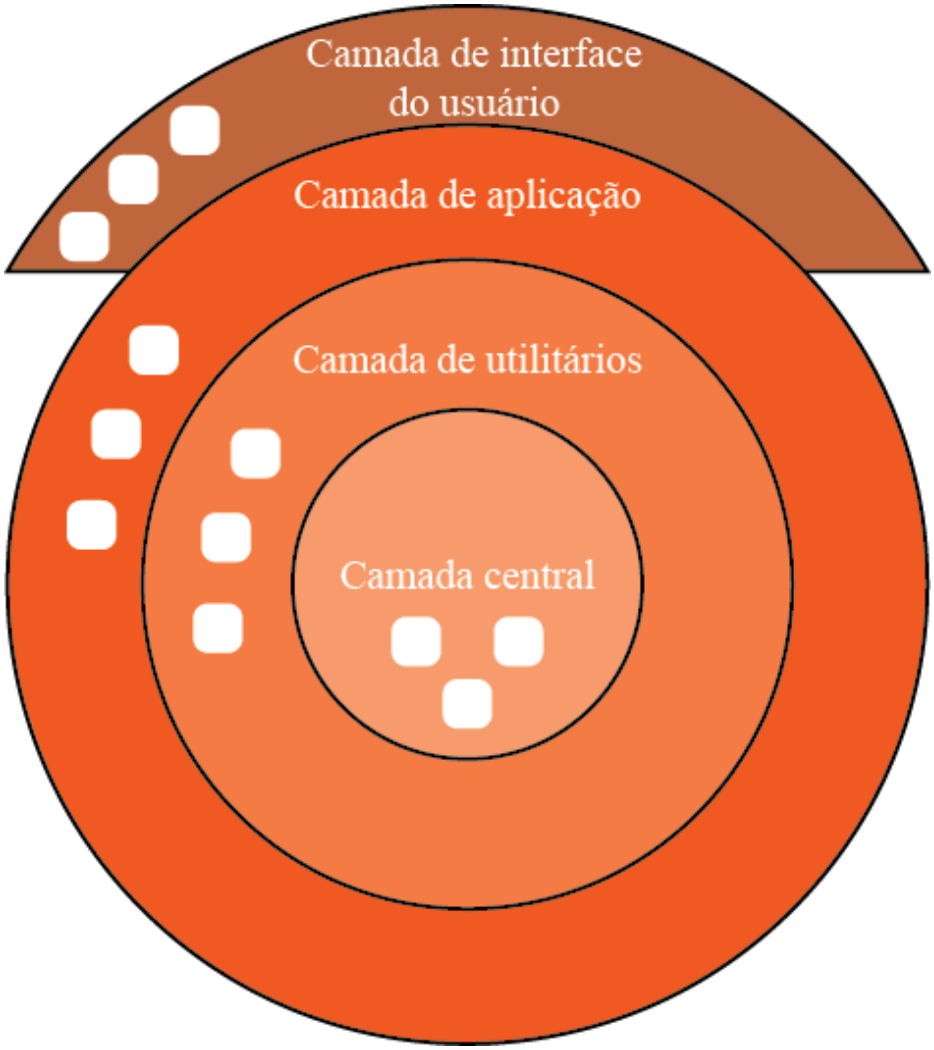


Figura 5 - Arquitetura em camadas demonstrando as diversas camadas e os componentes associados a elas. Fonte: Elaborada pelo autor, baseado em PRESSMAN, 2016.

#PraCegoVer: Ilustra a arquitetura em camadas demonstrando as diversas camadas e os componentes associados a elas. A imagem possui círculos, uns dentro dos outros, representando as camadas. Na camada mais interna está a Camada Central. Em direção mais externa, estão as camadas de utilitários, de aplicação e de interface do usuário, respectivamente.

Diante das características expostas das arquiteturas de *software*, é possível descrevermos os **padrões** arquiteturais.

- **MVC** (Modelo-Visão-Controlador): considerado a base do gerenciamento de interação, separa a apresentação e a interação dos dados do sistema, sendo que o sistema é estruturado em três componentes lógicos que se comunicam entre si: o componente **Modelo** é responsável por **gerenciar** o

sistema de dados e as operações associadas aos dados; o componente **Visão** é responsável por **definir** e **gerenciar** a forma de os dados se apresentarem; o componente **Controlador** é responsável por **gerenciar** a interação do usuário e passá-la para a Visão e o Modelo. Geralmente, este padrão pode interagir e visualizar os dados de diversas maneiras, sendo que sua vantagem é permitir que os dados podem ser alterados de forma independente da sua representação. Por outro lado, quando o modelo de dados é muito simples, pode envolver complexidade adicional desnecessária. Na figura a seguir, demonstraremos este modelo.

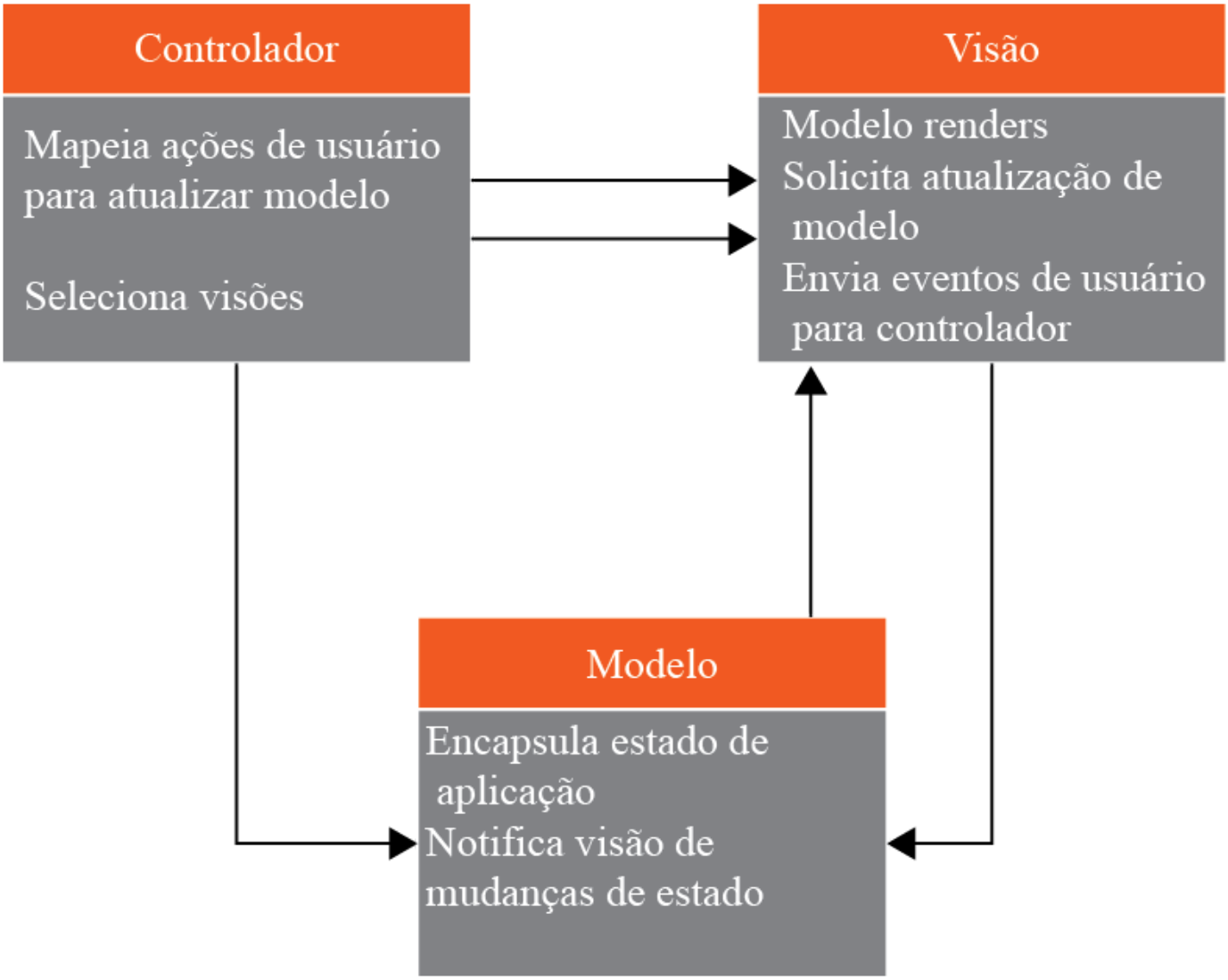


Figura 6 - Modelo da organização MVC com a demonstração do fluxo de ações e das mudanças de estado.

Fonte: Elaborada pelo autor, baseado em SOMMERVILLE, 2011.

#PraCegoVer: Ilustra o modelo da organização MVC com a demonstração do fluxo de ações e das mudanças de estado. A imagem possui três retângulos, Modelo, Controlador e Visão. O Modelo encapsula o estado de aplicação e notifica visão de

mudanças de estado. O controlador mapeia ações de usuário para atualizar o modelo e seleciona visões. A visão modelo os renders, solicita atualização de modelo e envia eventos de usuário para o controlador.

- **Abordagem em camadas**: sustenta o desenvolvimento de um sistema de forma incremental. Quando uma camada é desenvolvida, alguns serviços podem ficar disponíveis para os usuários. A arquitetura também tem como características a **mutabilidade** e a **portabilidade**. Se a interface ficar inalterada, uma camada pode ser substituída por outra equivalente, ou seja, quando uma camada de interfaces é alterada ou tem novos recursos incluídos, somente a camada adjacente é afetada. Portanto, este padrão se caracteriza pela organização do *software* em camadas, cuja funcionalidade é associada a cada uma delas. Desse modo, uma camada deve fornecer serviços à camada acima dela, e os níveis mais baixos de camadas representam os principais serviços utilizados no sistema. Sua vantagem é permitir a **substituição** de camadas inteiras, mas, por outro lado, é difícil ter clara a separação entre as camadas e como interagem camadas de níveis mais alto e mais baixo, impactando o desempenho geral do sistema.

VOCÊ QUER LER?

Existe uma proposta de arquitetura de *software* baseada no padrão em camadas e que demonstra sua aplicação na construção e integração de ambientes virtuais de aprendizagem. A arquitetura facilita a adição de novos módulos a estes sistemas de forma distribuída. Essa proposta foi realizada por Sizo; Lino e Favero (2010). Leia mais em: <http://www.scielo.mec.pt/scielo.php?script=sci_arttext&pid=S1646-98952010000200003 (http://www.scielo.mec.pt/scielo.php?script=sci_arttext&pid=S1646-98952010000200003)>.

- **De repositório**: tem como característica que todos os dados do sistema são gerenciados em um repositório central, ficando **acessível a todos** os componentes do sistema. Neste padrão, existe a vantagem de os componentes serem independentes. Mas, por outro lado, o repositório se torna um ponto único de falha.

- **Cliente-servidor**: é organizado segundo um conjunto de serviços e servidores ligados aos clientes. Segundo Sommerville (2011, p. 113, grifos nossos), os principais componentes deste modelo são:

  - Um conjunto de **servidores** que oferecem serviços a outros componentes. Exemplos de servidores incluem: servidores de impressão que oferecem serviços de impressão; servidores de arquivos que oferecem serviços de gerenciamento de arquivos; e um servidor de compilação, que oferece serviços de compilação de linguagens de programação.
  - Um conjunto de **clientes** que podem chamar os serviços oferecidos pelos servidores. Em geral, haverá várias instâncias de um programa cliente executando simultaneamente em computadores diferentes.
  - Uma **rede** que permite aos clientes acessar esses serviços. A maioria dos sistemas cliente-servidor é implementada como sistemas distribuídos, conectados através de protocolos de Internet.

Esse modelo geralmente é utilizado quando os dados compartilhados precisam ser acessados de vários locais. Na figura a seguir, demonstraremos esse modelo-padrão.

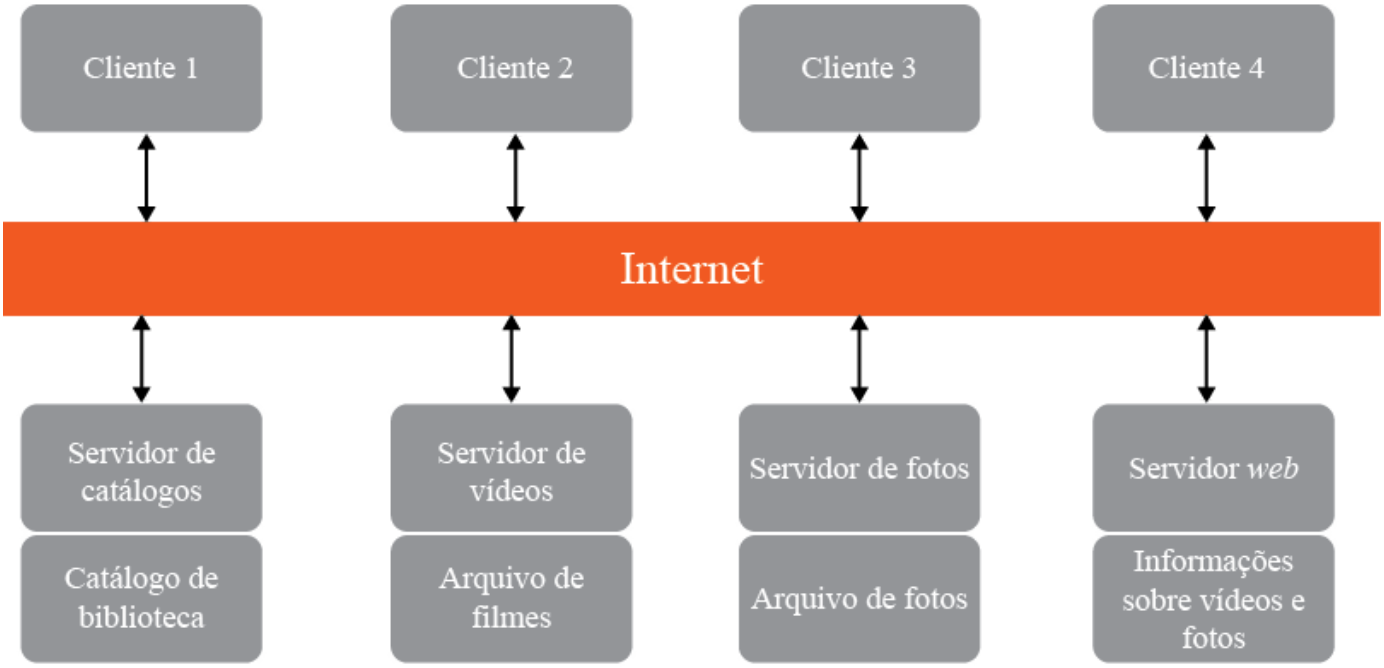


Figura 7 - Modelo cliente-servidor, demonstrando a relação entre clientes, internet e serviços disponíveis.

Fonte: Elaborada pelo autor, baseado em SOMMERVILLE, 2011.

#PraCegoVer: Ilustra o Modelo cliente-servidor, demonstrando a relação entre clientes, internet e serviços disponíveis. A imagem contém um grande retângulo com a inscrição Internet, acima estão 4 retângulos representando os clientes e

abaixo existem retângulos com as inscrições servidor de catálogos e catálogo de biblioteca para o cliente 1, servidor de vídeos e arquivo de filmes para o cliente 2, servidor de fotos e arquivos de fotos para o cliente 3 e, servidor web e Informações sobre vídeos e fotos para o cliente 4.

A vantagem desse modelo é a característica de **distribuição** por meio de uma rede, podendo estar disponível para todos os clientes. Porém, cada serviço se torna um ponto de falha, tornando o desempenho imprevisível.

VOCÊ SABIA?

O desenvolvimento de *software* baseado em agentes tem como objetivo a sua aplicação em sistemas distribuídos e visa buscar técnicas capazes de minimizar as dificuldades encontradas nessa nova abordagem, como demonstrado no trabalho desenvolvido por Huzita; Oliveira; Laine (2000). Veja mais a respeito em: <http://periodicos.uem.br/ojs/index.php/ActaSciTechnol/article/view/3131/2252 (http://periodicos.uem.br/ojs/index.php/ActaSciTechnol/article/view/3131/2252)>.

- **Dutos e filtros**: é um padrão de arquitetura no qual o processamento dos dados está organizado de forma que cada componente de processamento – o filtro – realize um determinado tipo de processamento dos dados, ou seja, os dados fluem – como um duto – de um componente para outro. Este tipo de arquitetura geralmente é utilizado em sistemas de **processamento de dados**, quando as entradas são processadas em **etapas distintas** para gerar saídas relacionadas entre si. Sua vantagem é a capacidade de **reuso** do processamento dos dados ser de fácil compreensão e facilidade de suporte. Por outro lado, o formato para a transferência dos dados deve ser acordado entre os processamentos.

1.3 Decisões sobre arquitetura

Os arquitetos de *software* criam uma **metodologia** própria para coletar e analisar os requisitos e definir a arquitetura para ser utilizada no **desenvolvimento** dos sistemas. Nesse sentido, são realizadas diversas questões que deverão ser

respondidas: como os usuários interagirão com o aplicativo e como ele será implantado e gerenciado? Quais são os requisitos de qualidade do aplicativo? Como projetá-lo para que seja de fácil manutenção? Quais são as tendências arquitetônicas que podem interferir no sistema? Podemos responder a essas questões considerando que um projeto de *software* desenvolvido com qualidade deve atender aos requisitos do usuário sem que haja interrupções e com uma usabilidade que atenda às demandas. Essas indagações afetam as decisões que o desenvolvedor tomará a respeito do *hardware*, dos componentes, das estruturas, das plataformas, dos sistemas de gerenciamento de *software*, entre outros recursos acoplados ao sistema e que ele deve se integrar.

Portanto, definir a arquitetura de *software* envolve implantar uma solução estruturada que atenda ao maior número possível dos **requisitos** técnicos e operacionais e que aperfeiçoe os atributos de **qualidade** (como desempenho, segurança e capacidade de gerenciamento). Decidir sobre ela significa analisar diversas **decisões**, que devem ser baseadas em vários fatores, e cada uma, provavelmente, trará um impacto considerável sobre o desempenho, a facilidade de manutenção e a qualidade do *software*.

CASO

Uma indústria de autopeças decidiu desenvolver internamente um sistema integrado de manutenção da fábrica com os departamentos de compras, controle de estoque e de contabilidade. Esse sistema é denominado de *Enterprise Resource Planning* (ERP), cujo objetivo principal é ter o máximo de integração dos processos disparando eventos entre os departamentos. Por exemplo, quando houver uma manutenção preventiva na fábrica, o sistema já executa o controle de estoque para garantir os insumos necessários para manutenção e dispara um evento para o departamento de compras e contabilidade, caso haja a necessidade de se adquirir algum insumo.

Os desenvolvedores estavam analisando qual seria o estilo de arquitetura que mais se adequaria ao modelo desse tipo sistema. Segundo os princípios analisados, as arquiteturas centralizada em dados, de fluxo de dados, orientada a objetos e em camadas não foram consideradas adequadas ao perfil do sistema. Concluiu-se que o estilo de chamadas e retornos seria o que mais se acopla à proposta de um ERP, devido ao fato de que esse estilo teria um programa principal: faria o controle de informações e procedimentos entre os outros subprogramas e procedimentos remotos.

As decisões de projeto de arquitetura incluem tanto aquelas do tipo de aplicação, distribuição do sistema e estilos da arquitetura que deverão ser utilizados quanto as formas de a arquitetura ser documentada e avaliada. Ou seja, além da escolha dos **algoritmos** e a forma da **estrutura de dados**, a arquitetura engloba decisões sobre as **estruturas** que formarão o sistema, o controle, os protocolos de comunicação, a sincronização e o acesso a dados, além da atribuição de funcionalidades aos elementos do sistema.

Outras variáveis a serem analisadas são: distribuição física dos elementos, escalabilidade, desempenho e demais atributos de qualidade. Assim, as tomadas de decisão necessitam de uma análise profunda e detalhada dos impactos, pois elas podem causar um **impacto** em todo o projeto. Assim sendo, o desenvolvedor deverá focar nos estudos da organização de forma global do sistema e seus relacionamentos entre subsistemas e demais componentes, utilizando ferramentas e técnicas, tais como visões gráficas e análise técnica.

1.3.1 Como as decisões sobre a arquitetura afetam o desenvolvimento de um sistema

Você sabe dizer como decisões específicas de arquitetura podem ser feitas enquanto o sistema é implantado? Como estas decisões impactam no desenvolvimento do sistema? O primeiro passo importante no processo de projeto é tomar a decisão de quais **modelos** de projeto que são necessários e o **nível de detalhamento** para eles. Isso dependerá do **tipo de sistema** que está sendo desenvolvido.

A maneira de projetarmos um sistema sequencial de processamento é diferente da maneira de fazer um sistema de tempo real – assim como a arquitetura terá modelos de projeto diferentes. Por exemplo, segundo Sommerville (2011, p. 130, grifos nossos), quando utilizamos a *Unified Modeling Language* (UML), geralmente utilizamos dois modelos de projeto:

- **estruturais**, os quais descrevem a estrutura estática do sistema, usando as classes de objetos e seus relacionamentos. Relacionamentos importantes que podem ser documentados nesse estágio são os de generalização (herança), usa/usado por, e composição.
- **dinâmicos**, os quais descrevem a estrutura dinâmica do sistema e mostram as interações entre os objetos do sistema. Interações que podem ser documentadas incluem a sequência de solicitações de serviço feitas pelos objetos e as mudanças de estado que são disparadas por essas interações de objetos.

De modo geral, as razões que levam às decisões do projeto original de arquitetura não são registradas. Porém, é necessário que os responsáveis pela evolução do sistema compreendam por que as decisões de projeto foram tomadas, sendo fundamental que elas sejam **documentadas**.

Os impactos tomados pela decisão de determinada arquitetura compreendem que as alterações propostas precisam ser muito bem analisadas. É importante ter em mente que, durante a implantação, pode haver a necessidade de se **reconsiderar as decisões** de projeto tomadas anteriormente. A partir do momento em que escolhemos a arquitetura de *software*, ela pode evoluir e, assim sendo, requer que novos requisitos arquiteturais sejam definidos com o objetivo de atender às mudanças desejadas. Portanto, devemos adicionar ou modificar características, e o sistema deverá estar preparado para estas mudanças.

VOCÊ O CONHECE?

John von Neumann foi um cientista que contribuiu fortemente para a evolução da Ciência da Computação, sendo um dos construtores do primeiro computador: o Eniac. Foi responsável pelo aprimoramento do poder computacional, levando-o a um aperfeiçoamento da estrutura lógica do processamento de dados, contribuindo de maneira inigualável para a evolução da Ciência da Computação. Veja mais a respeito no livro de Fonseca Filho (2007), disponível aqui (https://catalogcdns3.ulife.com.br/content-cli/CTI_ARQSOF_20/unidade_1/ebook/sections/pdf/historiadacomputacao.pdf).

Os impactos da escolha da arquitetura podem influenciar na **manutenibilidade** do sistema e na **adaptabilidade** dos requisitos funcionais e não funcionais. Outros fatores também podem significar o nível do impacto da decisão da arquitetura, como a capacidade de **escalabilidade** e a **refatoração** do sistema. Ou seja, neste tipo de desenvolvimento, ao invés de se desenvolver protótipos, a análise dos requisitos e a implantação ocorrem de forma conjunta. Os métodos ágeis são aqueles de desenvolvimento incremental, em que os incrementos são pequenos, e as novas versões do sistema são criadas e disponibilizadas aos clientes periodicamente. As versões envolvem os clientes no processo de desenvolvimento

para que se obtenham retornos rápidos sobre a evolução dos requisitos. Portanto, é possível perceber como a escolha da arquitetura pode ser fundamental para o **desenvolvimento de um sistema** de forma consistente e de alto desempenho.

Conforme demonstramos, cada decisão da arquitetura deve ser documentada para que possa haver uma revisão posterior, a fim de outros interessados entendam a descrição arquitetônica. Essa documentação deverá ser **sempre atualizada**, conforme os requisitos forem sendo alterados ou novos forem implantados.

1.4 Projeto de arquitetura

Você sabe qual é o objetivo e quais são as decisões que precisamos tomar ao fazer um projeto de arquitetura de um sistema? Entender e saber desenvolver um bom projeto de arquitetura é fundamental para o sucesso de um sistema.

O projeto de arquitetura é focado na compreensão de como um sistema de informação deverá ser organizado e como é estabelecida a estrutura global do sistema. No modelo do processo de desenvolvimento de um sistema, consideramos o projeto de arquitetura como o **primeiro estágio** do processo de projeto de *software*. Assim, uma arquitetura de *software* serve, fundamentalmente, como um **plano de projeto** para a negociação de requisitos de sistema com seus usuários e, em seguida, como uma forma de estruturar as discussões com os desenvolvedores e gerentes. É também uma ferramenta essencial que pode gerenciar a complexidade, pois pode esconder detalhes e permitir que os desenvolvedores foquem nas abstrações do sistema.

VOCÊ QUER VER?

O filme *O jogo da imitação* (TYLDUM; MOORE, 2014) mostra a história do matemático Alan Turing – considerado o pai da informática – e sua tentativa de quebrar um código de comunicação nazista durante a 2ª Guerra Mundial. O longa-metragem mostra um episódio fundamental na história da computação. Não deixe de conferir.

Quando é iniciado o projeto de arquitetura de um sistema, o *software* deverá ser dentro de um contexto que o represente, definindo as entidades externas que o sistema interage e o tipo da interação. Tais informações são obtidas pela **engenharia de requisitos** e que completam o modelo de representação do sistema no contexto.

1.4.1 Representação do sistema no contexto

Neste momento, podemos nos questionar: como os sistemas operam entre si? O que temos como resposta é o que chamamos de **contexto de arquitetura**, que representa como o sistema interage com entidades externas a seus limites. O desenvolvedor, para modelar como o sistema interage com suas entidades externas, utiliza o **Diagrama de Contexto Arquitetural**, e os sistemas que interagem com o chamado sistema-alvo são representados segundo Pressman (2016, p. 240, grifos nossos) como:

- **superiores** – sistemas que usam o sistema-alvo como integrante de algum esquema de processamento de mais alto nível.
- **subordinados** – sistemas que são utilizados pelo sistema alvo e fornecem dados ou processamentos necessários para completar a funcionalidade do sistema-alvo.
- **de mesmo nível** – sistemas que interagem em uma base par-a-par.
- **atores** – entidades que interagem com o sistema-alvo.

Poderemos representar a estrutura genérica do diagrama de contexto arquitetural na figura a seguir.

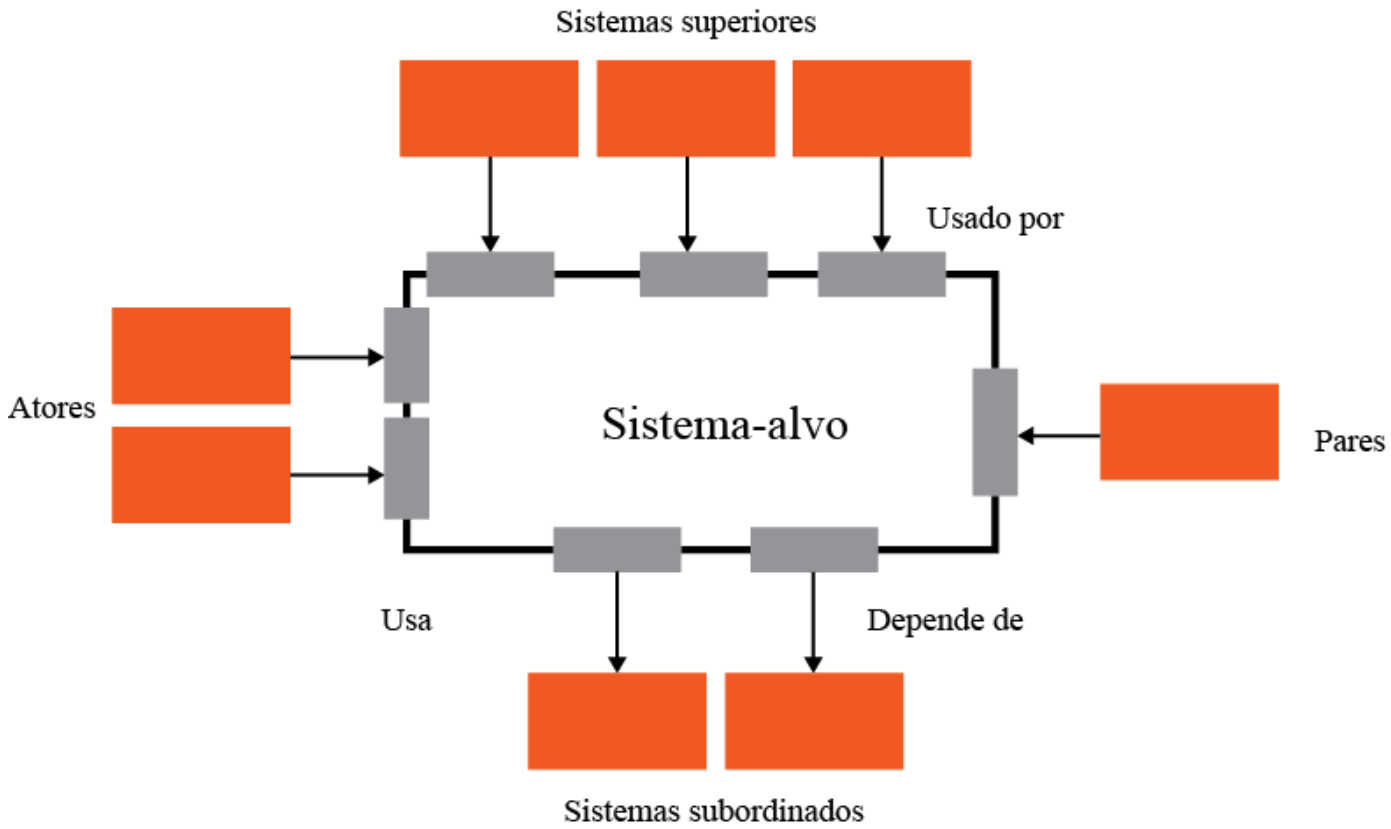


Figura 8 - Estrutura genérica do diagrama de contexto arquitetural mostrando os componentes e os relacionados hierárquicos. Fonte: Elaborada pelo autor, baseado em PRESSMAN, 2016.

#PraCegoVer: Ilustra a estrutura genérica do diagrama de contexto arquitetural mostrando os componentes e os relacionados hierárquicos. No centro da imagem há a inscrição sistema-alvo. Ao redor existem os demais componentes e sua relação hierárquica, onde sistemas superiores são usados por atores e pares que dependem de sistemas subordinados.

Cada uma das entidades externas, representadas no diagrama de contexto arquitetural, comunica-se com o sistema-alvo por meio de uma interface, mostrando como o fluxo e a direção dos dados interagem com os módulos e as entidades. Na figura a seguir, demonstraremos como acontece um Diagrama de Contexto em um sistema de controle de segurança.

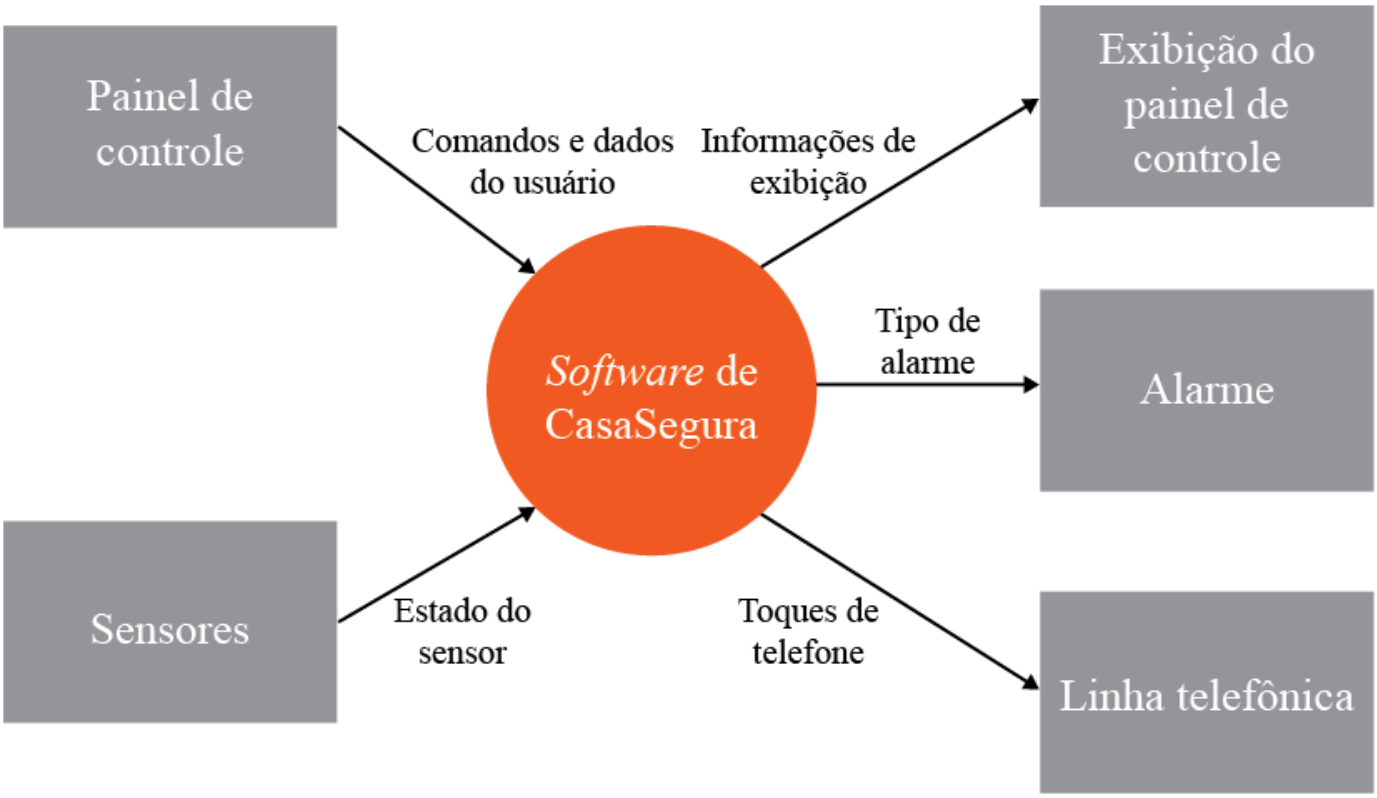


Figura 9 - Exemplo do Diagrama de Contexto em um sistema de segurança domiciliar. Fonte: Elaborada pelo autor, baseado em PRESSMAN, 2016.

#PraCegoVer: Ilustra um Exemplo do Diagrama de Contexto em um sistema de segurança domiciliar. No centro da imagem existe um círculo com a inscrição Software de Casa Segura. Dois retângulos, painel de controle e Sensores apontam para o círculo do software, que por sua vez, aponta para os retângulos Exibição do painel de controle, Alarme e Linha telefônica

Para produzirmos detalhes significativos, precisamos realizar o refinamento do diagrama e dos fluxos de dados, sendo que os **Diagramas de Fluxo de Dados** detalham estes refinamentos e estas transformações, a fim de exibir a coesão correspondente a cada transformação do refinamento.

1.4.2 Definição de arquétipos

Após ser modelado o diagrama de contexto, quais, podemos dizer, que seriam as próximas representações da arquitetura de *software*? Uma vez que o contexto é modelado e as interfaces foram representadas, devemos identificar o conjunto de arquétipos arquiteturais, isto é, uma forma abstrata do que seria a representação de um **elemento do *software***.

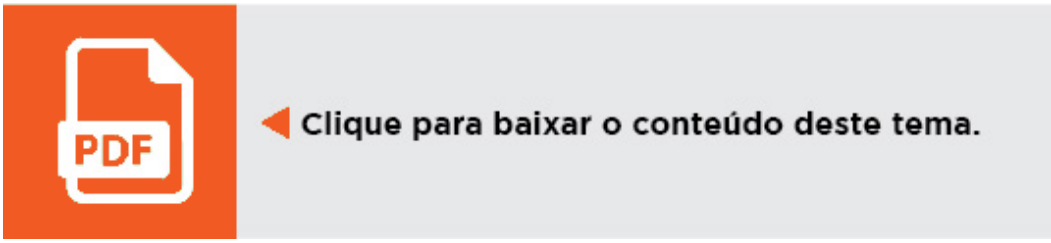
A maioria dos sistemas é representada por um número pequeno de arquétipos, pois a arquitetura do sistema-alvo é composta destes arquétipos, que são elementos **estáveis**, e são derivados após a análise de as classes serem definidas no modelo de requisitos. Portanto, os arquétipos auxiliam no **desenvolvimento** do projeto da arquitetura de *software*, levando-o a um nível de **detalhamento** que torna mais fácil detectar inconsistências entre os componentes arquiteturais.

Síntese

Compreendemos, com o fim este capítulo, o quanto é importante elaborarmos as arquiteturas de *software* para o desenvolvimento de projetos de sistemas e tornar possível o desenvolvimento das soluções de sistemas de forma mais simples, rápida, com melhor eficiência e fácil manutenibilidade. Ao implantarmos um determinado modelo de arquitetura de *software*, devemos atingir ao máximo a simplicidade, para que o sistema tenha as características de ser flexível, extensível, portável e reutilizável.

Neste capítulo, você teve a oportunidade de:

- compreender os princípios de arquitetura de *software*;
- entender as fases de desenvolvimento de um projeto arquitetural;
- descrever os elementos pertinentes à arquitetura e aos padrões arquiteturais básicos;
- compreender os tipos de modelagem e a utilização dos diferentes estilos, padrões e gêneros na especificação da arquitetura de *software*;
- analisar os impactos das decisões do tipo de arquitetura no desempenho de um sistema.



Bibliografia

CAGNIN, R. L. **Uma arquitetura multiagente para gerenciamento de dispositivos em ambientes da internet das coisas**. 2015. 123 f. Dissertação (Mestrado em Ciência da Computação) – Universidade Estadual Paulista, São José do Rio Preto, 2015. Disponível em: <http://www.athena.biblioteca.unesp.br/exlibris/bd/cathedra/02-08-2017/000868971.pdf (http://www.athena.biblioteca.unesp.br/exlibris/bd/cathedra/02-08-2017/000868971.pdf)>. Acesso em: 15/5/2018.

CUNHA, D.; FILGUEIRA, A; VIANA, G. Arquitetura de software voltada para a avaliação contínua do processo de ensino-aprendizagem. **Revista HOLOS**, Natal, IF-RN, ano 33, v. 1, p. 43-56, 2017. Disponível em: <http://www2.ifrn.edu.br/ojs/index.php/HOLOS/article/view/5335/pdf (http://www2.ifrn.edu.br/ojs/index.php/HOLOS/article/view/5335/pdf)>. Acesso em: 15/5/2018.

FONSECA FILHO, C. **História da computação**: o caminho do pensamento e da tecnologia. Porto Alegre: PUC-RS, 2007. Disponível em: <http://www.pucrs.br/edipucrs/online/historiadacomputacao.pdf (http://www.pucrs.br/edipucrs/online/historiadacomputacao.pdf)>. Acesso em: 15/5/2018.

GALLOTI, G. M. A. **Arquitetura de software**. São Paulo: Pearson Education do Brasil, 2016.

HOFMEISTER, C.; NORD, R.; SONI, D. **Applied Software Architecture**. Boston: Addison-Wesley, 2000.

HUZITA, E. H. M.; OLIVEIRA, H. M.; LAINE, J. M. Uma proposta de arquitetura de software baseada em agentes. **Acta Scientiarum** - Technology, Maringá, UEM, v. 22, n. 5, p. 1339-1346, 2000. Disponível em: <http://periodicos.uem.br/ojs/index.php/ActaSciTechnol/article/view/3131/2252 (http://periodicos.uem.br/ojs/index.php/ActaSciTechnol/article/view/3131/2252)>. Acesso em: 15/5/2018.

PRESSMAN, R. **Engenharia de software**: uma abordagem profissional. 8. ed. Porto Alegre: McGraw Hill, 2016.

SIZO, A. M.; LINO, A. D. P.; FAVERO, E. L. Uma proposta de arquitetura de software para construção e integração de ambientes virtuais de aprendizagem. **Revista Ibérica de Sistemas e Tecnologias de Informação (RISTI)**, Porto, n. 6, p. 17-30, dez.

2010. Disponível em: <http://www.scielo.mec.pt/pdf/rist/n6/n6a03.pdf (http://www.scielo.mec.pt/pdf/rist/n6/n6a03.pdf)>. Acesso em: 15/5/2018.

SOMMERVILLE, I. **Engenharia de software**. 9. ed. São Paulo: Pearson Prentice Hall, 2011.

TYLDUM, M.; MOORE, G. **O jogo da imitação**. Direção: Morten Tyldum. Produção: Morten Tyldum. Estados Unidos: Warner Bros, 2014.